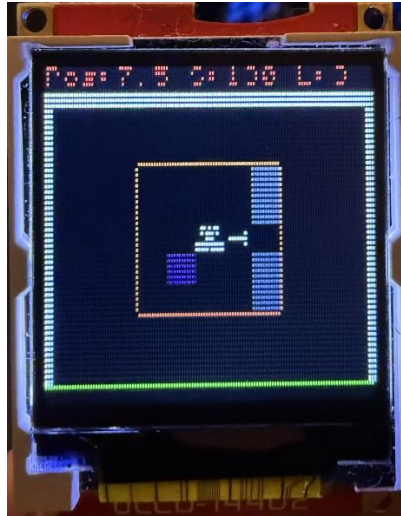


ECE 2035: Project 2-2

Cave explorer!

The goal of this project is to build a 2D exploration game using the Mbed platform. The theme is up to you! We will refer to things in the context of a cave exploration game, but feel free to make the theme your own: deep sea, space, dungeon, desert, or anything else you come up with. In this project, you will build a handheld game using the provided shell code, your HW3 circuit, and your hash table implementation from P2-1.



In your game, you will be controlling an explorer character, whose goal is to traverse the map, collect powerups and tools, defeat enemies, and modify the landscape to earn points to defeat the final boss and escape the cave. The movement of the explorer will be controlled using the nav switch and the pushbuttons will be used for advanced features. The map area will be much larger than what can be displayed on the screen, which is where your hash table comes in. You will use the hash table to store the locations of objects on the map, and you will look up their correct locations to display them at each game update.

There are several basic features that your game must implement to receive baseline credit; advanced features are an opportunity to earn full and possibly extra credit. The list of basic features and examples of some advanced features are given below.

Please note that the game features, art, and actions available in the game are intentionally vague, and the specific design of the game is up to your creativity. Games can be extremely varied, and this project is an opportunity for you to make something unique. HAVE FUN WITH IT!

Explorer Motion, Map, Items

The explorer moves around in the direction pressed on the nav switch. The explorer is a struct that holds previous and current x and y positions along with other information such as powerup status. The definition for the explorer struct can be found in explorer.h.

The map for your game is the world that your explorer moves around in. It is made up of individual tiles, with a total width of 40 tiles and a height of 25 tiles, not all of which can be displayed on the screen at one time. As you navigate the map, your explorer should always be displayed in the center of the screen. We've gotten you started with code for existing walls to create a more structured starting map, but you can feel free to add or remove them as desired, as long as your map maintains a similar overall complexity. The only hard requirement is that you leave the boss chamber in the top right walled-off as is!

ECE 2035: Project 2-2

You will need to populate the Map with *Items* relevant to your game - these items can include scenery, walls, plants, powerups, tools, etc. Please note that the explorer object itself is not a MapItem. Implementation details for the Map are discussed in detail in the [technical reference](#) section of this document, below.

The Goal

The object of the game is to explore the map to gain enough points and powerups to defeat the boss and escape the cave! The game consists of the following steps.

1. Start the game somewhere on the screen (your choice, as long as you don't start in the boss chamber in the top right). There should be boostup/down objects scattered on the screen along with a complex wall structure.
2. Since your explorer is stuck in a dark and mysterious environment, they should start with low visibility around themselves, with only part of the screen being rendered.
3. The user should then navigate around the map to collect different powerups (such as ones to increase visibility), traverse uncharted tiles, and gather tools to prepare to fight the boss. All of these actions should grant points.
4. Gain as many boosts and points as possible before challenging the boss!
5. When ready, unlock the portal to the boss chamber and fight the boss.
6. After defeating the boss or running out of lives, display a *game over* screen and end the game.

Graphics

A *status* area is set aside at the top of the screen. The status area should display (at a minimum) the current score and your explorer's coordinates. This area can also be used to show other information related to advanced features.

As a baseline feature, you must implement at least one *sprite*. A *sprite* in this context is a tile that's more than just a rectangle, and also not something as simple as a single character of ASCII text. You should be using the provided `draw_img(...)` or `uLCD.BLIT(...)` functions to accomplish this. Example sprites are provided in `graphics.cpp`.

Basic Feature Summary

These features are for baseline credit and constitute a functional, but minimal, cave explorer game. Each is worth 4pts, up to a total of 40pts.

1. Nav-switch moves the explorer in all 4 directions. (4pt)
2. Status bar shows explorer coordinates, the current score (default 0), and the number of lives (default 1). (4pt)
3. The score increases by 1 when we successfully move to a tile (only if we are not blocked by a wall or other map item). (4pt)
4. Night vision tool placement (4pt)
 - a. Any interactive item like this may **not** be placed on top of a pre-existing item (wall, other tool, etc.)
 - b. A constant number of night vision tools (3, 5, 10, etc.) should be placed randomly when the game starts. After this, a new one should be placed randomly each time the explorer collects an existing one. Hint: seed your random number generator with the time it takes the user to start the game with a button press.
5. Night vision tool functionality: when collected, the player should gain visibility of the full uLCD screen for a limited time, after which we revert to low visibility. Their score should also increment by +10. (4pt)
6. Poison and heal items: place at least 3 of each item randomly throughout the map each time the game starts. If the player walks into a poison item, subtract 1 life; if they walk into a healing item, add 1 life. (4pt)
7. Create and use a sprite of your own design! (4pt)
 - a. You cannot use one of the provided sprites in `graphics.cpp` for this feature.
8. Teleporter: implement a teleporter that appears near your initial spawn point once the player has reached 100pts. The teleporter tile should place the player in the boss chamber when the player steps onto it. (4pt)

ECE 2035: Project 2-2

9. Win condition: End game and display a win screen if the explorer presses the center of the nav-switch after stepping through the teleporter to the boss chamber. (4pt)
10. Loss condition: End game and display a game over screen when we run out of lives (i.e. lives == 0). (4pt)

Advanced Feature Examples

Advanced features in this project are open ended and allow you to make the game your own. The features listed below are all acceptable, but this is not an exhaustive list. Each extra feature is worth +5 points unless stated otherwise, and you must tell your grader at the beginning of your demo which extra features you used. Other features that are at or above the difficulty level of those listed here are acceptable *but must be approved by an instructor or graduate TA*: there will be a pinned discussion topic on Ed Discussion for this purpose. If you intend to use features that aren't listed here, start a follow-up on this discussion thread to clear it with the GTAs or instructors before grading begins. *Note also that any feature mentioning a pushbutton can also use the "center-press" of the nav switch if desired.*

- **Boss fight:** Now that you can teleport into the boss chamber, finish what you started and design a boss fight! The boss should be confined to the chamber and move independently of the player, with some randomness. It should take several attacks to defeat and be able to strike back with its own attack (subtracting lives). Some ideas: it could have a ranged lance, a shockwave attack around itself, or a projectile attack, or anything else you can think of. Instead of displaying the win screen just by reaching the chamber, win the game when you've defeated the boss! To encourage you to try this out, **this feature will be worth 10pts.**
- **Tools and weapons (each 1 feature):** To defeat the boss or reshape the map, you'll need tools. For each, the tool or weapon should be spawned at a random location on the map at the start of the game, with a new copy of that tool appearing at a new location when the player picks one up. After pickup, it should grant +5 score when used and should be held in front of the player (1 tile ahead, adjusting dynamically in response to movement direction).
 - **Pickaxe:** If the pickaxe overlaps with a wall tile and a pushbutton of your choice is pressed, the wall tile should be destroyed. Note that outer bordering walls and boss chamber walls must be immune to pickaxe hits.
 - **Sword and shield:** The sword and shield can be used to damage the boss or any other enemy targets in the tile in front of the player, depending on the last direction of motion. It can also be used to block incoming damage from in front of the player. Both of these effects should be active only when a pushbutton is pressed.
 - **Bow and arrow:** The bow and arrow can be used to damage the boss and other enemies up to 3 tiles in front of the player, depending on the last direction of motion. It is fired by pressing a pushbutton of your choosing.
 - **Shovel:** The shovel can remove harmful items from the map, such as poison items. If it overlaps with a harmful item, press the chosen pushbutton to remove that item.
- **Tool inventory:** If the player picks up a new tool, it should typically just replace the current one they have, if any. Implement a system to track which tools have been collected and allow the player to cycle through tools by pressing a given pushbutton.
- **Shockwave attack:** upon collecting a randomly placed item, give the player 1 use of a shockwave attack that damages enemies and items in a set radius around them in response to a pushbutton. Note that boss chamber walls and outer boundary walls must be immune.
- **Roaming enemies:** Add a handful of enemies that roam randomly through the cave, subtracting 1 life on contact with the explorer. All attacks that can be used on the boss can be used on the enemies.
- **Sprites:** Have 5 or more additional objects represented with sprites to make your game look awesome. Check out <https://www.piskelapp.com> to make your sprites that you can import and draw on the screen using the `draw_img(...)` or `uLCD.BLIT(...)` methods.

ECE 2035: Project 2-2

- **Possibility to lose and start over:** allow the game to be replayed from the beginning without requiring a reset of the Mbed.
- **Game state management:** Use a pushbutton to pause and resume the game, preserving game state.
- **Game start page:** Add some form of user interaction to your start page. Examples include allowing the user to toggle to an instructions screen, allowing the user to select different sprite options for their explorer, allowing the user to toggle to a ‘controls’ screen, etc.
- **Sound effects:** You can add background music using the speaker from your kit. Suggestions include: start screen music, win music, game over music, etc.
- **Directional Sprites:** Create sprites for the explorer or any other moving entity that change with the direction of movement (e.g. if your explorer just walked left, their sprite should face left). You could also implement directional sprites for any tools held by your player. Sprites used in implementing this feature may be used for the “Sprites” feature as well.
- **Map highlighting:** Highlight any tile that you’ve already walked with a marking MapItem. Make the score only increment for each unique tile traversed, rather than for each successful traversal.
- **Animation:** Animation for interactions with things in the map (e.g., wall shattering if destroyed with pickaxe, boss defeat animation, etc.). The animation must consist of at least 3 different frames.
- **Wrap-around walls:** Rather than colliding with the outer boundary walls, the explorer can destroy them with a modified pickaxe to break through and wrap around to the other side of the map when they reach an edge, on the same row/column that they walked off from. Note that this requires the pickaxe tool be implemented.
- **Leaderboard:** Track the high score across multiple plays and display it on the start menu. Highscore does **not** need to be preserved after resetting the mbed.

This list is **NOT** comprehensive. Please feel free to ask about additional features on Ed Discussion, where we will maintain a list of approved advanced features!!

P2-2 Technical Reference

In this project, you’ll be combining hardware interface libraries for an LCD screen, pushbuttons, nav-switch, and possibly the speaker into a cohesive game. The shell code to support all this implementation has several different modules. This document is intended to be a reference for various technical considerations you’ll need when implementing your game.

Hash Table

The game will make use of your HashTable library, implemented in P2-1. In order to use this library within the Mbed environment, the easiest strategy is to simply copy and paste the code into the correct files. The shell project has two files already for this purpose: *hash_table.cpp* and *hash_table.h*. Copy your completed code from P2-1 into these files before starting anything else. Then implement the functions in *map.cpp* to create and access the HashTable used to implement the map functions (as described in the Map Module section below).

USB Serial Debug

Debugging is an important part of any software project and it can be more difficult with embedded systems. Fortunately, there is a built-in serial monitor on the Mbed that allows you to see printf-style output from the Mbed on your computer. For debugging any of your Mbed code, make use of the printf() function! As long as your Mbed is connected to your computer and you have the PlatformIO Serial Monitor open in VS Code, you should be able to see the terminal output at the bottom of your screen.

Game Loop Overview

The basic structure for organizing a video game is called the *game loop*. Each iteration of this loop is known as a *frame*. At each frame, the following operations are performed, typically in this order:

ECE 2035: Project 2-2

1. Read inputs
2. Update game state based on the inputs
3. Draw the game
4. Frame delay

You'll be implementing parts of each of these steps. The game loop shell code (with timing already implemented) is in *main.cpp*. Comments included in the game loop indicate what you need to implement. Refer to *.h* files for specification and documentation of functions. Be sure to add documentation and comments for the code you write.

Read inputs. Reading user input for a frame happens only once during the frame. This serves two purposes. For one, it isolates the part of the code that works with input hardware to just the `read_inputs` function, allowing the rest of the code to deal with only the results of the input operation. Secondly, it ensures a constant value of the “true” input for a particular frame. If there are multiple parts of your game update logic that must interact with the inputs, it is convenient to know that these inputs are guaranteed to be the same.

Update game. This is where the magic happens. Based on the current state of the game -- where the explorer is, whether it a boost-up/boost-down is in effect, what other objects are nearby -- you compute what the next state should be, based on the inputs you've already measured. For example, if you moved the nav switch to the up position, the explorer should move up in the map. This is where most your development will be focused.

Draw game. With the state updated, you now need to show the user what changed by drawing it to the screen. This step is discussed in much more detail below, but for now you'll want to know that the entry point to this portion of the code is called `draw_game`.

Frame delay. By default, loops in C run as fast as the instructions can possibly execute. This is great when you're trying to sort a list, but it's really bad for games! If the game updates as fast as possible, the user might not be able to understand what's happening or control the character appropriately. So, we introduce a delay that aims to make each frame take 100ms. The time for all the proceeding 3 steps is measured, and the remaining time is waited out before starting again. If more than 100ms has already passed, no additional delay is added. As you're developing your game, be careful that your frames don't get too long, or the feel of your game will degrade.

Map Module

In order to think about updating the game state (moving the player) and drawing the screen, we first need some way to represent the world. This module accomplishes that task.

The map is a two-dimensional grid whose origin is at the top-left corner of the world. The X coordinate increases toward the right, and the Y coordinate increases toward the bottom. This left-handed coordinate system is chosen for consistency with the graphics; see that section for more details. The finest granularity of the map is a single grid tile; the player moves from tile to tile, and each tile contains at most one `MapItem`. If the tile is empty, then that tile is free space on the map.

The map in this game is represented by a collection of `MapItems` held in a `HashTable`. The keys in the `HashTable` are (x,y) pairs. The data in the `HashTable` are all of type `MapItem`, defined in *map.h*. So, for example, if you access the key “(10, 23)” in the `HashTable` and the data is a `MapItem` whose type is `WALL`, then the explorer should not be able to walk into that tiles.

The shell code is written so that the use of the `HashTable` is hidden inside the map module; that is, the *hash_table.h* is only included from *map.cpp*, and the `HashTable` functions are only used internally to that module. The public API of the map module does not expose the `HashTable` since this is an implementation detail of the map and does not affect the rest of the game. This hides the complexity of the `HashTable` (questions like “what is the best hash function?” and “how do I map (x,y) pairs into integer keys for use in my hash table?”) within the map module itself and simplifies the rest of the game logic.

ECE 2035: Project 2-2

The public API for the map module is provided in *map.h*. All functions and structures are documented there. There are functions for accessing items (e.g., `get_here`, `get_north`) and modifying the map (e.g., `add_wall`, `map_erase`). *You are encouraged to add more functionality to this API as you deem necessary for your game.* The point of an API is to be useful to the programmer; if these functions are insufficient, add more!

Map Items. This is the basic unit of the map and is the underlying type of all the void* data in the map HashTable. Each MapItem has an integer field, called “type” which tells you what kind of item it is. This allows you to store different information in the map, such as the location of walls and the location of trees, using the same data structure. Each MapItem also has a function pointer of type DrawFunc that will draw that MapItem. Its inputs are a pixel location (u,v) of the tile. Finally, each MapItem has two additional parameters: an integer flag, walkable, that describes if the explorer is allowed to walk on that tile; and a void* data for storage of any extra data required during the game update. Walls probably don’t need extra data; other items with more interesting interactions might; it is up to your implementation.

Two-dimensional keys. As you implemented in P2-1, the HashTable accepts only unsigned integers as keys. However, for this application you need to use two integers (the X & Y coordinates) as the key. In order to do this, you need to have a function to map these coordinates unambiguously into a single integer. This function is called XY_KEY, and is private to *map.cpp*. You then also need a hash function that will take this key as normal and produce a hash value for bucket selection in the HashTable.

Graphics

The graphics module houses most of the drawing code for the game. This includes all the drawing functions for the various MapItems. The entry point for drawing the screen is the `draw_game` function. This section describes how that function accomplishes drawing the tiles, and various ideas to consider as you extend this function for your own game.

Coordinate Reference Frames. There are several relevant coordinate frames for this game. The first we have already covered: the map frame. This frame’s coordinates are labeled (x,y) and its origin is the top corner of the map. X increases right, and Y increases down. All frames in the game are this left-handed orientation.

The next frame is the local drawing frame. This frame is centered on the explorer, and ranges from (-5,-4) to (5,4), i.e. it is an 11x9 grid of tile. This frame is iterated in `draw_game` and each tile is drawn in turn using the DrawFunc from the map, or a `draw_nothing` function if there is no MapItem. The coordinates of this frame are labeled (i,j).

Finally, there are the pixels on the screen. This frame has its origin at the top-left corner of the screen. The screen is a 128x128 array of pixels. The coordinates of this frame are labelled (u,v). Each tile in the map is 11x11 pixels.

Drawing functions. As noted above, each MapItem has an associated DrawFunc that knows how to draw that item. These functions take as input a (u,v) coordinate for the top-left pixel of the tile and draw an 11x11 image that represents the MapItem. An example is the `draw_wall` function, in *graphics.h*. You will need to add more draw functions as you add more MapItem types. You are free to implement these in whatever way you like, using the full power of the uLCD library. However, a simple way to do this has been given to you. The `draw_img` function takes a string of 121 (= 11 * 11) characters, each representing a pixel color, and translates that into a BLIT command to draw those colors to the screen. You can use this function to make nice graphics very simply by defining a new string that represents the image you want to draw. This is the recommended method for generating art for your game. Feel free to use <https://www.piskelapp.com> to design your sprites and assist with converting them to strings.

Drawing Performance. The screens are notoriously slow to draw, and the length of time it takes to complete a drawing command is proportional to the number of pixels that it changes on the screen. So, the drawing code goes through some hoops to make sure that things keep moving quickly. In particular, the drawing code requires not only the current explorer position (`explorer.loc_x` and `explorer.loc_y`) but also the previous position (`explorer.loc_x_prev` and `explorer.loc_y_prev`), in order to determine what has changed on the screen. If an element on the screen has not

ECE 2035: Project 2-2

changed, it does not need to be redrawn. This saves time and makes the game update more quickly. You'll need to be careful with this as you decide how many items to put in your map and how to draw the new items you add.

You must design, implement, and test your own code. There are many, many ways to code this project, and many different possibilities for timing, difficulty, responsiveness and general feel of the game. Your project should represent your interpretation of how the game should feel and play. Any submitted project containing code (other than the provided framework code and mbed libraries) not fully created and debugged by the student constitutes academic misconduct.

In addition, the materials provided by the instructor in this course are for the use of the students currently enrolled in the course. Copyrighted course materials may not be further disseminated. Project code files must not be made publicly available anywhere.